

An Analysis of:

A New Modified Sorting Algorithm:

A Comparison with State of the Art



Adam **Kearns** ⁽²³³²⁶⁹⁰³⁾, Cormac **Kehoe** ⁽²³⁴¹¹³⁰²⁾, William **Lin** ⁽²³³¹⁴²¹³⁾, Liam **Maher** ⁽²³⁴⁴⁴⁸⁸²⁾
University College Dublin, Dublin, Ireland

Introduction

PROPOSED GOAL

This project explores a proposed new efficient sorting algorithm that challenges some traditional comparative based sorts, in the hopes of answering these fundamental questions:

- How can we sort arrays containing negative values without negative array indexing?
- Is there an efficient way to avoid $O(n \log n)$ comparisons seen in modern, state of the art sorting algorithms?
- And if so, is it possible to maintain a near linear time performance regardless of the input?

KEY IDEA

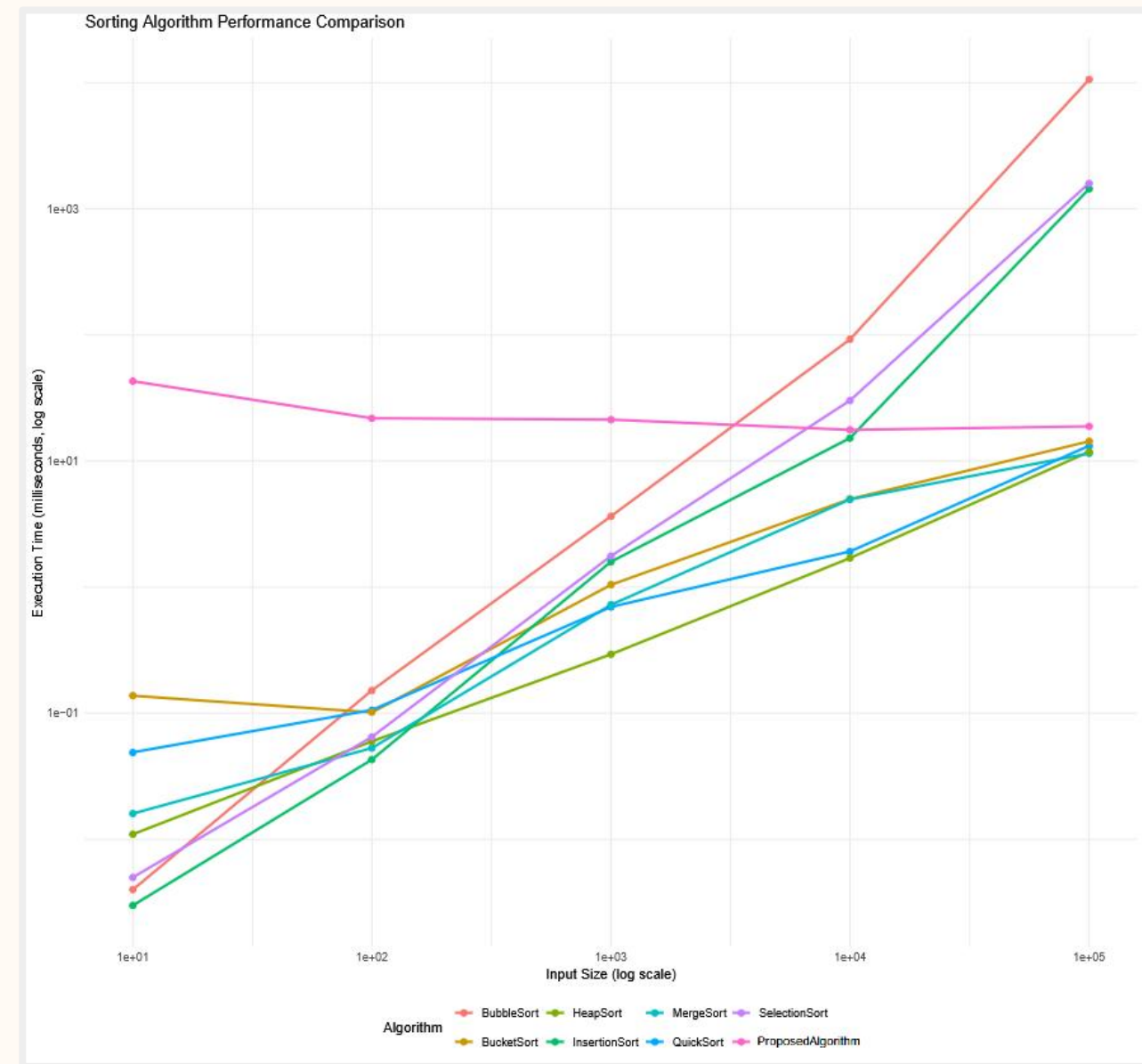
By instead using a combination of *counting-based* logic and *offset indexing* to include negative values, we can split elements into bins using there value position:

IMPLEMENTATION

- Determine *min/max* values of the array to establish the range.
- Iterate over array and increment values corresponding index in its respective bin (positive, negative and zero elements). This step is similar to that of countSort.
- Starting with the smallest negative value, check how many times it appears and write it that many times to the result array. Continue for all negative values.
- We then check how many times zero appears, and again write however many times it does appear into the result array.
- And finally, for positive numbers, our output will be similar to negatives, without offsetting the index

★ Our resulting array is a sorted array of both positive, negative and zero values.

Expanded Functionality



INTERPRETATION OF RESULTS

Although the paper only tests the proposed algorithm with a range of values $-8 \leq x \leq +8$, we decided to go one step further and test with a full range of values from $-10^6 \leq x \leq 10^6$, and again ran the same tests to experiment and see what interesting results we could find. We realised the following:

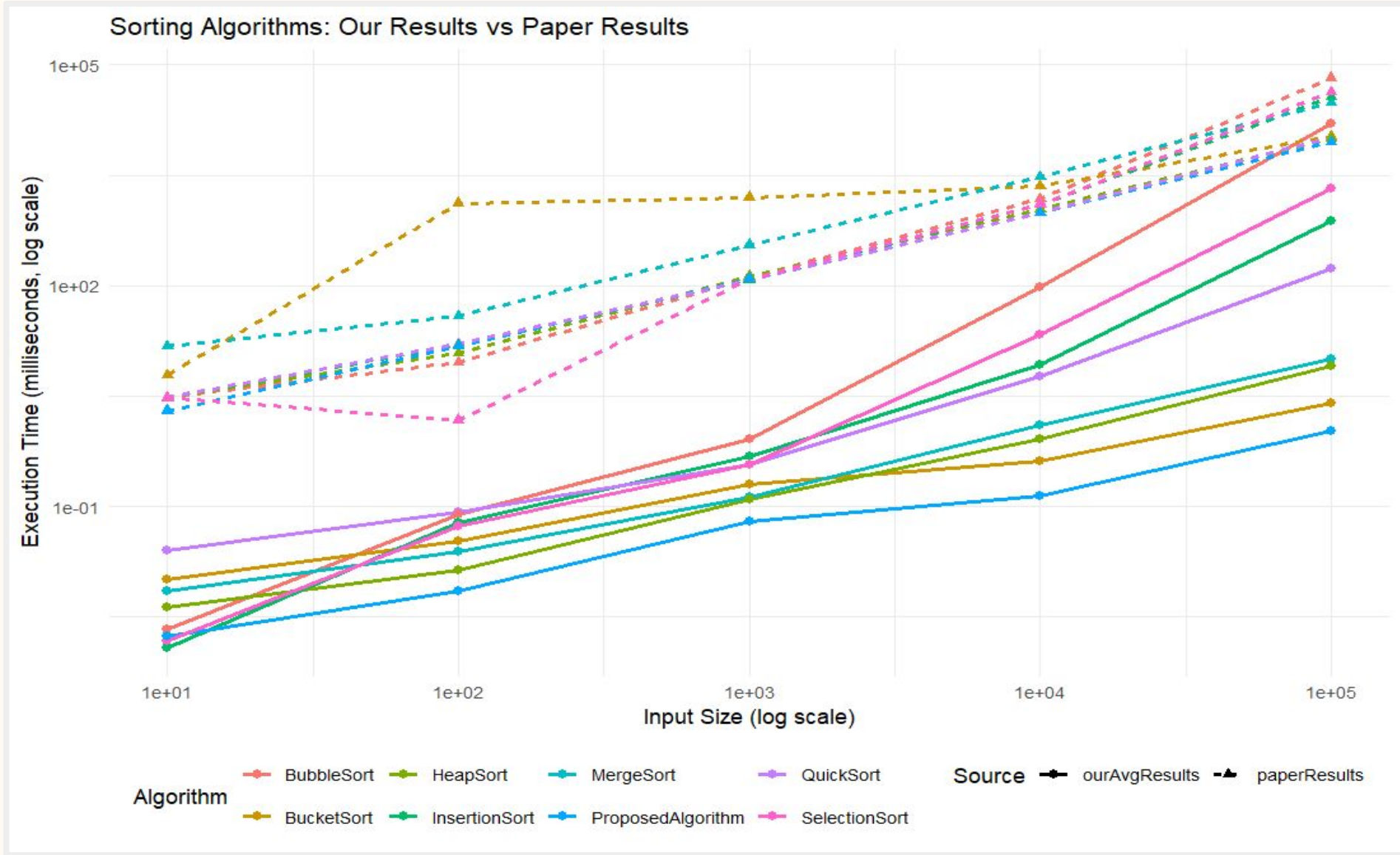
- The popular sorting algorithms were definitely a lot fast than the proposed algorithm for a small array size (10, 100, 1000 elements). However they grew exponentially slower as the input size increased, as shown in the graph.
- However, interestingly, the proposed algorithm seemed to stay almost constant for any array input size (pink line on graph).

See FURTHER RESULTS DISCUSSION for further development ideas

#Inputs	ProposedAlg()	QuickSort()	BubbleSort()	BucketSort()	MergeSort()	HeapSort()	InsertionSort()	SelectionSort()
10	0.0017	0.0256	0.002133...	0.010066...	0.007166...	0.0043	0.0012	0.001466...
100	0.007133...	0.08266...	0.0779	0.033733...	0.024066...	0.013533...	0.0595	0.054233...
1000	0.062833...	0.367266...	0.834	0.1991	0.136566...	0.124933...	0.474366...	0.365266..
10000	0.13866...	5.9192	94.88566...	0.415566...	1.279066...	0.82866...	8.276233...	21.499233...
100000	1.058033...	172.8136	16077.92317	2.5786	10.0765	8.138933...	766.2407	2093.949033

Results above are in milliseconds (ms)

Comparative Results Analysis

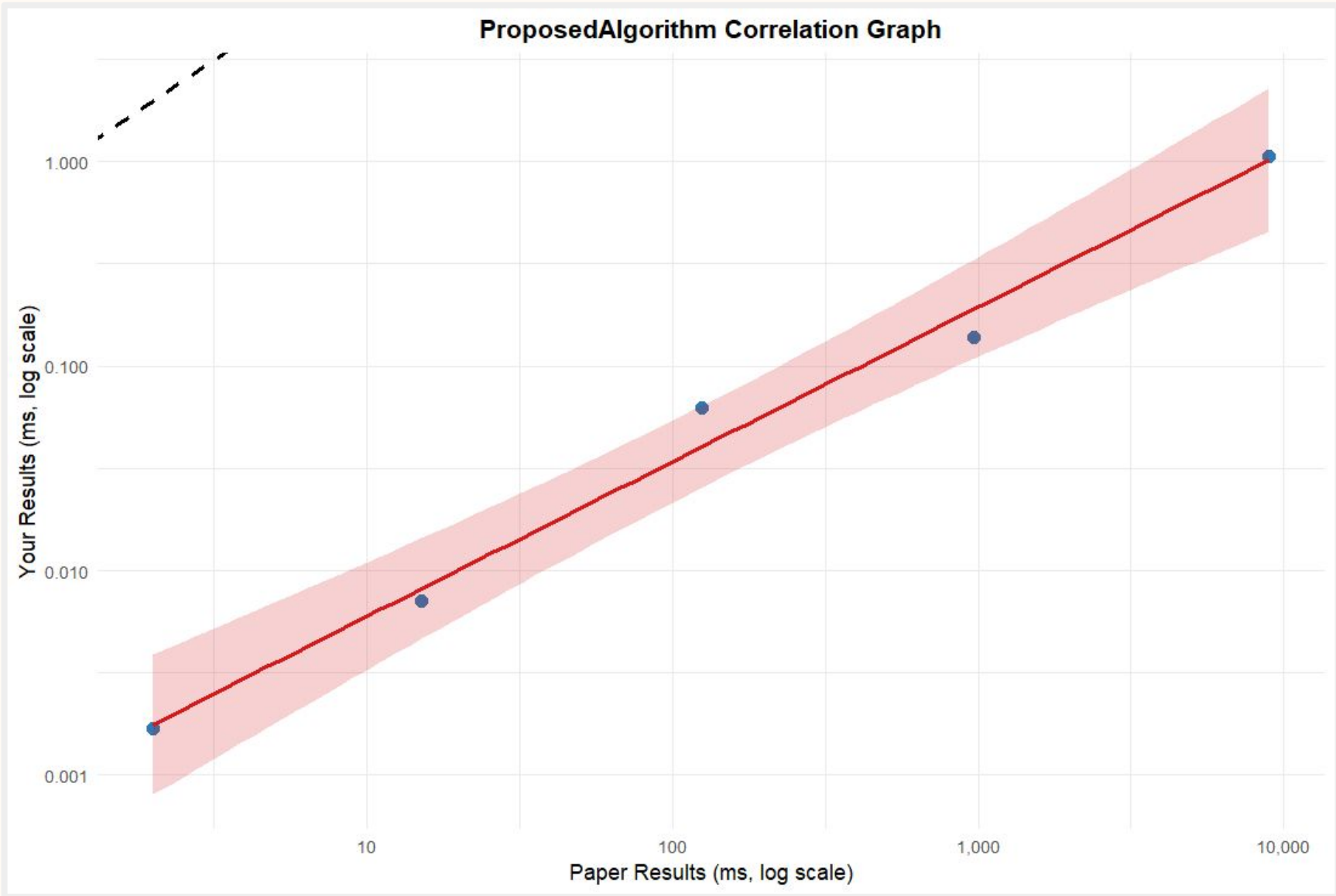


KEY FINDINGS

- Our results closely match the trends reported in the paper.
- Our execution times are lower.
- ProposedAlgorithm outperformed all other algorithms, roughly after input size of 30.
- Time Complexity of $O(n + k)$.
- Effective in a range of -8 to +8 integers.
- QuickSort, MergeSort, BucketSort and ProposedAlgorithm show good performance for large datasets.
- Scales well and is efficient

KEY FINDINGS

- Strong correlation between our results and paper's results
- Regression analysis shows strong linear relationship(on a log to log scale)
- Our regression points lie within the confidence interval (95% confidence).
- Linear dashed line represents a perfect 1:1 relationship between our results and the paper's results.



Further Results Discussion

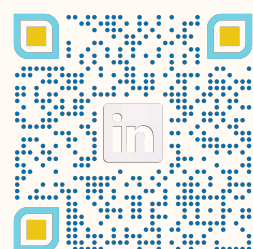
FURTHER DEVELOPMENT IDEAS

With the information we gained earlier (see EXPANDED FUNCTIONALITY for more context), we alluded to the idea that for a very large input amount of possibly high digit numbers ($-10^8 \leq x \leq 10^8$) this proposed algorithm could offer a very time efficient sort relative to "State of the Art" sorts.

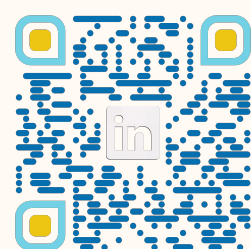
We propose that this could prove very useful for sorting very long decimal numbers (by splitting the pre and post decimal digits into 2 separate sorts). However, due to the fact that the required amount of arrays would be doubled, we concluded that the space complexity would prove it impractical for general use.

References & Resources

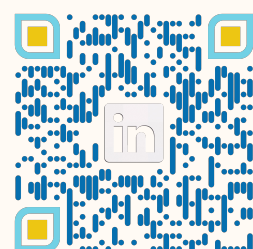
- "A New Modified Sorting Algorithm: A Comparison with State of the Art" ^[2017]
- Graphing Software - R studio
- IDE & Language - IntelliJ IDEA w/Java



Liam



Adam



Cormac



William